

Knapsack01 thief memoization

```
#include <bits/stdc++.h>

using namespace std;

long long solveMemo(int i, int w, const vector<int> &wt, vector<int> &val, vector<vector<long long>>&dp)
{
    //base case
    if(i==0)
    {
        if(wt[0]<=w)
            return val[0];
        else
            return 0LL;
    }

    if(dp[i][w]!=-1) return dp[i][w];

    long long notPick= solveMemo(i-1,w,wt,val,dp);

    long long pick= LLONG_MIN/4;
    if(wt[i]<=w)
    {
        pick= val[i]+ solveMemo(i-1,w-wt[i],wt,val,dp);
    }

    return dp[i][w]= max(pick,notPick);
}

long long knapsack01(const vector<int>& wt, vector<int>& val, int W) {
    int n = (int)wt.size();
    vector<vector<long long>> dp(n, vector<long long>(W + 1, -1));
    return solveMemo(n - 1, W, wt, val, dp);
}

int main()
{
    int n,W;
    cin>>n>>W;
    vector<int>wt(n), val(n);

    for(int i=0;i<n;i++)
    {
        cin>>wt[i];
    }
    for(int i=0;i<n;i++)
    {
        cin>>val[i];
    }

    cout<<knapsack01(wt,val,W)<<endl;
}
```

Knapsack01 thief tabulation

```
#include <bits/stdc++.h>

using namespace std;

long long knapsack01(const vector<int>& wt, vector<int>& val, int W)
{
    int n= (int)wt.size();
    vector<vector<long long>> dp(n,vector<long long>(W+1,0));

    //base case
    for(int w=0;w<=W;w++)
    {
        dp[0][w]= (wt[0]<=w) ? val[0]:0LL;
    }

    for(int i=1;i<n;i++)
    {
        for(int w=0;w<=W;w++)
        {
            long long notPick= dp[i-1][w];
            long long pick= notPick;
            if(wt[i]<=w)
            {
                pick= max(pick,(long long) val[i]+dp[i-1][w-wt[i]]);
            }
            dp[i][w]=pick;
        }
    }

    return dp[n-1][W];
}

int main()
{
    int n,W;
    cin>>n>>W;
    vector<int>wt(n), val(n);

    for(int i=0;i<n;i++)
    {
        cin>>wt[i];
    }
    for(int i=0;i<n;i++)
    {
        cin>>val[i];
    }

    cout<<knapsack01(wt,val,W)<<endl;
}
```

0.1 minimum_{coin_infinite_m}memoization

```
// Returns minimum number of coins to make 'T', or INF if impossible.
int memoSolve(int i, int T, const vector<int>& coins, vector<vector<int>>& dp) {
    const int INF = 1e9;
```

```
// Base: only coin[0] is available
if (i == 0) {
    if (T % coins[0] == 0) return T / coins[0];
    return INF;
}

if (dp[i][T] != -1) return dp[i][T];

// Not pick coin i
int notPick = memoSolve(i - 1, T, coins, dp);

// Pick coin i (unbounded -> stay at i)
int pick = INF;
if (coins[i] <= T) {
    int sub = memoSolve(i, T - coins[i], coins, dp);
    if (sub != INF) pick = 1 + sub;
}

return dp[i][T] = min(pick, notPick);
}

int minCoins_memo(const vector<int>& coins, int amount) {
    int n = (int)coins.size();
    vector<vector<int>> dp(n, vector<int>(amount + 1, -1));
    const int INF = 1e9;

    int ans = memoSolve(n - 1, amount, coins, dp);
    return (ans >= INF) ? -1 : ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, amount;
    cin >> n >> amount;
    vector<int> coins(n);
    for (int i = 0; i < n; ++i) cin >> coins[i];

    cout << minCoins_memo(coins, amount) << "\n";
    return 0;
}
```

0.2 minimum_{coin_infinite_t}abulation

```
#include <bits/stdc++.h>
using namespace std;

// Returns minimum number of coins to make 'T', or INF if impossible.
int memoSolve(int i, int T, const vector<int>& coins, vector<vector<int>>& dp) {
    const int INF = 1e9;

    // Base: only coin[0] is available
    if (i == 0) {
        if (T % coins[0] == 0) return T / coins[0];
        return INF;
```

```

}

if (dp[i][T] != -1) return dp[i][T];

// Not pick coin i
int notPick = memoSolve(i - 1, T, coins, dp);

// Pick coin i (unbounded -> stay at i)
int pick = INF;
if (coins[i] <= T) {
    int sub = memoSolve(i, T - coins[i], coins, dp);
    if (sub != INF) pick = 1 + sub;
}

return dp[i][T] = min(pick, notPick);
}

int minCoins_memo(const vector<int>& coins, int amount) {
    int n = (int)coins.size();
    vector<vector<int>> dp(n, vector<int>(amount + 1, -1));
    const int INF = 1e9;

    int ans = memoSolve(n - 1, amount, coins, dp);
    return (ans >= INF) ? -1 : ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, amount;
    cin >> n >> amount;
    vector<int> coins(n);
    for (int i = 0; i < n; ++i) cin >> coins[i];

    cout << minCoins_memo(coins, amount) << "\n";
    return 0;
}

```

0.3 Dice Combinations

```

const int MOD = 1e9 + 7;
vector<int> dp;

//memoization
int solve(int n) {
    if (n == 0) return 1;    // one valid way (no dice)
    if (n < 0) return 0;    // invalid
    if (dp[n] != -1) return dp[n];

    long long ways = 0;
    for (int k = 1; k <= 6; k++) {
        ways += solve(n - k);
        ways %= MOD;
    }
    return dp[n] = ways;
}

```

```

}

//tabulation
int diceWays(int n) {
    vector<int> dp(n + 1, 0);
    dp[0] = 1; // base case: 1 way to make sum 0 (no dice)

    for (int x = 1; x <= n; x++) {
        long long ways = 0;
        for (int k = 1; k <= 6; k++) {
            if (x - k >= 0) {
                ways += dp[x - k];
            }
        }
        dp[x] = ways % MOD;
    }

    return dp[n];
}

int main() {
    int n;
    cin >> n;
    dp.assign(n + 1, -1);

    cout << solve(n) << "\n";
    return 0;
}

```

0.4 Coin Combination 1 and 2

```

const int MOD = 1e9 + 7;

// Order matters (permutations)
int countPermutations(int n, int x, const vector<int>& coins) {
    vector<int> dp(x + 1, 0);
    dp[0] = 1;

    for (int sum = 1; sum <= x; sum++) {
        long long ways = 0;
        for (int coin : coins) {
            if (sum - coin >= 0) {
                ways += dp[sum - coin];
                if (ways >= (long long)MOD * 4) ways %= MOD;
            }
        }
        dp[sum] = (int)(ways % MOD);
    }
    return dp[x];
}

// Order does not matter (combinations)
int countCombinations(int n, int x, const vector<int>& coins) {
    vector<int> dp(x + 1, 0);
    dp[0] = 1;

    for (int coin : coins) {

```

```

        for (int sum = coin; sum <= x; sum++) {
            dp[sum] += dp[sum - coin];
            if (dp[sum] >= MOD) dp[sum] -= MOD;
        }
    }
    return dp[x];
}

int main() {
    int n, x;
    cin >> n >> x;
    vector<int> coins(n);
    for (int i = 0; i < n; i++) cin >> coins[i];

    cout << "Permutations (order matters): "
         << countPermutations(n, x, coins) << "\n";

    cout << "Combinations (order doesnt matter): "
         << countCombinations(n, x, coins) << "\n";

    return 0;
}

```

0.5 DP: Counting Tiling

```

#include <bits/stdc++.h>
using namespace std;
const int MOD = 1e9+7;

int n, m;

// recursive helper: try filling column row by row
void dfs(int row, int mask, int nextMask, int n, vector<int>& nextMasks) {
    if (row == n) {
        nextMasks.push_back(nextMask);
        return;
    }
    if (mask & (1<<row)) {
        // this row is already filled, move on
        dfs(row+1, mask, nextMask, n, nextMasks);
    } else {
        // vertical domino: occupies current row in both masks
        dfs(row+1, mask|(1<<row), nextMask|(1<<row), n, nextMasks);
        // horizontal domino: occupy current row and next row ( if free)
        if (row+1 < n && !(mask & (1<<(row+1)))) {
            dfs(row+2, mask|(1<<row)|(1<<(row+1)), nextMask, n, nextMasks);
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> m;
}

```

```

int states = 1<<n;

// precompute transitions for each mask
vector<vector<int>> trans(states);
for (int mask=0; mask<states; mask++) {
    vector<int> nextMasks;
    dfs(0, mask, 0, n, nextMasks);
    trans[mask] = nextMasks;
}

// DP table: only need two columns at a time
vector<int> dp(states, 0), ndp(states, 0);
dp[0] = 1;

for (int col=0; col<m; col++) {
    fill(ndp.begin(), ndp.end(), 0);
    for (int mask=0; mask<states; mask++) {
        if (dp[mask] == 0) continue;
        for (int nxt : trans[mask]) {
            ndp[nxt] = (ndp[nxt] + dp[mask]) % MOD;
        }
    }
    dp.swap(ndp);
}

cout << dp[0] << "\n";
return 0;
}

```

0.6 Divide and Conquer: merge sort

```

void merge(vector<int>& arr, int l, int m, int r) {
    vector<int> temp;
    int i = l, j = m + 1;

    // merge by picking the smaller from left/right
    while (i <= m && j <= r) {
        if (arr[i] <= arr[j]) temp.push_back(arr[i++]);
        else temp.push_back(arr[j++]);
    }
    // append leftovers
    while (i <= m) temp.push_back(arr[i++]);
    while (j <= r) temp.push_back(arr[j++]);

    // copy back
    for (int k = 0; k < (int)temp.size(); ++k)
        arr[l + k] = temp[k];
}

void mergeSort(vector<int>& arr, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(arr, l, m);
    mergeSort(arr, m + 1, r);
    merge(arr, l, m, r);
}

int main() {
    int n;

```

```

    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    mergeSort(a, 0, n - 1);

    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << a[i];
    }
    cout << '\n';
    return 0;
}

```

0.7 maximum sub array sum

```

#include <bits/stdc++.h>
using namespace std;

int maxCrossingSum(const vector<int>& arr, int l, int m, int r)
{
    // Find max suffix sum in left half
    long long sum = 0, leftSum = LLONG_MIN;
    for (int i = m; i >= l; i--) {
        sum += arr[i];
        leftSum = max(leftSum, sum);
    }

    // Find max prefix sum in right half
    sum = 0;
    long long rightSum = LLONG_MIN;
    for (int i = m + 1; i <= r; i++) {
        sum += arr[i];
        rightSum = max(rightSum, sum);
    }

    return (int)(leftSum + rightSum);
}

int maxSubarraySum(const vector<int>& arr, int l, int r) {
    if (l == r) return arr[l];

    int m = l + (r - l) / 2;

    int left = maxSubarraySum(arr, l, m);
    int right = maxSubarraySum(arr, m + 1, r);
    int cross = maxCrossingSum(arr, l, m, r);

    return max({left, right, cross});
}

int maxSubarraySum_Kadane(const vector<int>& arr) {
    int n = arr.size();
    long long curr_sum = arr[0]; // start with first element
    long long best_sum = arr[0]; // global max also starts here

    for (int i = 1; i < n; i++) {
        if (curr_sum + arr[i] < arr[i]) {

```

```

            curr_sum = arr[i]; // better to start new subarray
        } else {
            curr_sum += arr[i]; // extend current subarray
        }
        best_sum = max(best_sum, curr_sum); // update global max
    }
    return (int)best_sum;
}

int main() {
    int n;
    cin >> n;
    vector<int> arr(n);
    for (int i = 0; i < n; i++) cin >> arr[i];

    cout << maxSubarraySum(arr, 0, n - 1) << "\n";
    return 0;
}

```

0.8 Longest_{nice} substring

```

#include <iostream>
#include <vector>
#include <set>
#include <unordered_set>

using namespace std;

bool isNice(const string &s)
{
    unordered_set<char> st(s.begin(), s.end());
    for(char ch: st)
    {
        if(isupper(ch) && !st.count(tolower(ch)))
            return false;
        if(islower(ch) && !st.count(toupper(ch)))
            return false;
    }
    return true;
}

string longestNice(string s)
{
    if(s.length() < 2)
    {
        return "";
    }

    if(isNice(s))
        return s;

    for(int i=0 ; i<s.length();i++)
    {
        char ch=s[i];
        bool hasLower= (s.find(tolower(ch))<s.length());
        bool hasUpper = (s.find(toupper(ch))< s.length());

```

```

    if(!hasLower || !hasUpper)
    {
        string left= longestNice(s.substr(0,i));
        string right= longestNice(s.substr(i+1));
        return (left.length()>=right.length()) ? left:right;
    }
}
return "";
}

```

```

int main()
{
    string s;
    cin>>s;
    cout<<longestNice(s)<<endl;
    return 0;
}

```

0.9 Convex Hull

```

#include <bits/stdc++.h>
using namespace std;

```

```

struct Point {
    long long x, y;
};

```

```

// Cross product of OA OB
long long cross(const Point &O, const Point &A, const Point &B)
{
    return (A.x - O.x) * (B.y - O.y) -
           (A.y - O.y) * (B.x - O.x);
}

```

```

vector<Point> convexHull(vector<Point> pts) {
    int n = pts.size(), k = 0;
    if (n <= 1) return pts;

    // Sort points lexicographically (by x, then y)
    sort(pts.begin(), pts.end(), [](auto &a, auto &b) {
        return (a.x == b.x) ? a.y < b.y : a.x < b.x;
    });
}

```

```

vector<Point> H(2*n);

// Build lower hull
for (int i = 0; i < n; i++) {
    while (k >= 2 && cross(H[k-2], H[k-1], pts[i]) <= 0) k--;
    H[k++] = pts[i];
}

```

```

// Build upper hull
for (int i = n-2, t = k+1; i >= 0; i--) {
    while (k >= t && cross(H[k-2], H[k-1], pts[i]) <= 0) k--;
    H[k++] = pts[i];
}

```

```

H.resize(k-1);
return H;
}

int main() {
    int n;
    cin >> n;
    vector<Point> pts(n);
    for (int i = 0; i < n; i++) cin >> pts[i].x >> pts[i].y;

    auto hull = convexHull(pts);

    cout << "Convex Hull points:\n";
    for (auto &p : hull)
        cout << p.x << " " << p.y << "\n";

    return 0;
}

```

0.10 Minimal Euclidean Distance

```

#include <bits/stdc++.h>
using namespace std;

```

```

struct Point {
    long long x, y;
};

```

```

long long dist2(const Point &a, const Point &b) {
    long long dx = a.x - b.x;
    long long dy = a.y - b.y;
    return dx*dx + dy*dy;
}

```

```

long long bruteForce(vector<Point>& pts, int l, int r) {
    long long ans = LLONG_MAX;
    for (int i = l; i <= r; i++) {
        for (int j = i+1; j <= r; j++) {
            ans = min(ans, dist2(pts[i], pts[j]));
        }
    }
    return ans;
}

```

```

long long closestUtil(vector<Point>& pts, int l, int r) {
    if (r - l + 1 <= 3) {
        return bruteForce(pts, l, r);
    }
}

```

```

int mid = (l + r) / 2;
long long midx = pts[mid].x;

```

```

long long d = min(closestUtil(pts, l, mid),
                  closestUtil(pts, mid+1, r));

```

```

// Build strip of points within sqrt(d) distance from mid
line
vector<Point> strip;

```

```

for (int i = l; i <= r; i++) {
    if ((pts[i].x - midx)*(pts[i].x - midx) < d)
        strip.push_back(pts[i]);
}

// Sort strip by y
sort(strip.begin(), strip.end(), [](auto &a, auto &b){
    return a.y < b.y;
});

// Check each point with next few points
for (int i = 0; i < (int)strip.size(); i++) {
    for (int j = i+1; j < (int)strip.size() &&
         (strip[j].y - strip[i].y)*(strip[j].y - strip[i].y)
         < d; j++) {
        d = min(d, dist2(strip[i], strip[j]));
    }
}

return d;
}

```

```

long long closestPair(vector<Point>& pts) {
    sort(pts.begin(), pts.end(), [](auto &a, auto &b){
        return a.x < b.x;
    });
    return closestUtil(pts, 0, pts.size()-1);
}

```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<Point> pts(n);
    for (int i = 0; i < n; i++) {
        cin >> pts[i].x >> pts[i].y;
    }

    cout << closestPair(pts) << "\n";
    return 0;
}

```

0.11 Lab 5: Roads in Berland

```

#include <iostream>
#include <vector>
using namespace std;

```

```

// Floyd Warshall to compute all pairs shortest paths initially
void floydWarshall(int n, vector<vector<long long>>& dist){
    for(int k=0;k<n;k++){
        for(int i=0;i<n;i++){
            for(int j=0;j<n;j++){
                if(dist[i][k]+dist[k][j]<dist[i][j])
                    dist[i][j]=dist[i][k]+dist[k][j];
            }
        }
    }
}

```

```
// Update dist matrix and sumDist after adding a new edge (a,b,
c)
long long updateWithNewEdge(int n, vector<vector<long long>>&
dist, int a, int b, long long c, long long sumDist){
    if(c>dist[a][b]) return sumDist;
    for(int i=0;i<n;i++){
        for(int j=i+1;j<n;j++){
            long long newDist=min(dist[i][a]+c+dist[b][j], dist[
i][b]+c+dist[a][j]);
            if(newDist<dist[i][j]){
                sumDist+=(dist[i][j]-newDist);
                dist[i][j]=newDist;
                dist[j][i]=newDist;
            }
        }
    }
    return sumDist;
}
```

```
int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n; cin>>n;
    vector<vector<long long>> dist(n,vector<long long>(n));
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            cin>>dist[i][j];
        }
    }
    floydWarshall(n,dist);
    long long sumDist=0;
    for(int i=0;i<n;i++){
        for(int j=i+1;j<n;j++){
            sumDist+=dist[i][j];
        }
    }
    int k; cin>>k;
    while(k--){
        int a,b; long long c;
        cin>>a>>b>>c;
        a--; b--;
        sumDist=updateWithNewEdge(n,dist,a,b,c,sumDist);
        cout<<sumDist<<" ";
    }
    return 0;
}
```

0.12 interval_scheduling(max_n on – overlapping)

```
#include <bits/stdc++.h>
using namespace std;

struct Interval {
    int start, finish;
};

int maxNonOverlapping(vector<Interval>& intervals) {
    // sort by finish time
    sort(intervals.begin(), intervals.end(),
        [](auto &a, auto &b){ return a.finish < b.finish; });

    int count = 0;
    int last_finish = INT_MIN;
```

```
for (auto &iv : intervals) {
    if (iv.start >= last_finish) {
        count++;
        last_finish = iv.finish;
    }
}
return count;
}

int main() {
    int n;
    cin >> n;
    vector<Interval> intervals(n);
    for (int i=0; i<n; i++)
        cin >> intervals[i].start >> intervals[i].finish;

    cout << maxNonOverlapping(intervals) << "\n";
}
```

0.13 job_sequencing(max_profit)

```
#include <bits/stdc++.h>
using namespace std;

struct Job {
    int id;
    int deadline;
    long long profit;
};

// Greedy Job Sequencing returning profit + chosen job sequence
pair<long long, vector<int>> jobSequencing(vector<Job>& jobs) {
    int maxD = 0;
    for (auto &job : jobs) maxD = max(maxD, job.deadline);

    // Sort jobs by profit descending
    sort(jobs.begin(), jobs.end(), [](const Job& a, const Job&
b){
        return a.profit > b.profit;
    });

    vector<int> slot(maxD + 1, -1); // slot[t] = job id at time
t
    long long totalProfit = 0;

    for (auto &job : jobs) {
        for (int t = min(job.deadline, maxD); t >= 1; --t) {
            if (slot[t] == -1) {
                slot[t] = job.id;
                totalProfit += job.profit;
                break;
            }
        }
    }

    // Extract scheduled jobs (ignore -1)
    vector<int> sequence;
    for (int t = 1; t <= maxD; t++) {
```

```
        if (slot[t] != -1) sequence.push_back(slot[t]);
    }

    return {totalProfit, sequence};
}

int main() {
    int n;
    cin >> n;
    vector<Job> jobs(n);
    for (int i = 0; i < n; i++) {
        cin >> jobs[i].deadline >> jobs[i].profit;
        jobs[i].id = i + 1; // give each job an id
    }

    auto [profit, sequence] = jobSequencing(jobs);

    cout << "Max Profit: " << profit << "\n";
    cout << "Job Sequence: ";
    for (int id : sequence) cout << id << " ";
    cout << "\n";

    return 0;
}
```

0.14 interval_partitioning(min_resource)

```
#include <bits/stdc++.h>
using namespace std;

struct Interval {
    int start, finish, id;
};

// Function for interval partitioning
pair<int, vector<int>> intervalPartitioning(vector<Interval>&
jobs) {
    // 1) Sort jobs by start time
    sort(jobs.begin(), jobs.end(), [](auto &a, auto &b){
        return a.start < b.start;
    });

    // 2) Min-heap: (finish time, resource id)
    priority_queue<pair<int,int>, vector<pair<int,int>>,
        greater<>> pq;

    int resourceCount = 0;
    vector<int> assignment(jobs.size() + 1); // assignment by
job.id

    for (auto &job : jobs) {
        if (!pq.empty() && pq.top().first <= job.start) {
            // Reuse resource
            int res = pq.top().second;
            pq.pop();
            assignment[job.id] = res;
            pq.push({job.finish, res});
        } else {
            // Allocate new resource
```

```

        resourceCount++;
        int res = resourceCount;
        assignment[job.id] = res;
        pq.push({job.finish, res});
    }

    return {resourceCount, assignment};
}

int main() {
    int n;
    cin >> n;
    vector<Interval> jobs(n);
    for (int i = 0; i < n; i++) {
        cin >> jobs[i].start >> jobs[i].finish;
        jobs[i].id = i + 1;
    }

    auto [resources, assignment] = intervalPartitioning(jobs);

    cout << "Minimum resources needed: " << resources << "\n";
    cout << "Assignments:\n";
    for (int i = 1; i <= n; i++) {
        cout << "Job " << i << " Resource " << assignment[i] <<
            "\n";
    }

    return 0;
}

```

0.15 DC: Stick length

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<long long> p(n);
    for (int i = 0; i < n; i++) cin >> p[i];

    sort(p.begin(), p.end());
    long long median = p[n/2]; // middle element

    long long cost = 0;
    for (long long x : p) cost += llabs(x - median);

    cout << cost << "\n";
    return 0;
}

```

0.16 two_sets

```

#include <bits/stdc++.h>

```

```

using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
    cin >> n;
    long long total = n * (n + 1) / 2;

    if (total % 2 != 0) {
        cout << "NO\n";
        return 0;
    }

    cout << "YES\n";
    vector<int> set1, set2;
    long long target = total / 2;

    // Greedy: fill set1 until reaching target
    for (int i = n; i >= 1; i--) {
        if (target >= i) {
            set1.push_back(i);
            target -= i;
        } else {
            set2.push_back(i);
        }
    }

    cout << set1.size() << "\n";
    for (int x : set1) cout << x << " ";
    cout << "\n";

    cout << set2.size() << "\n";
    for (int x : set2) cout << x << " ";

    cout << "\n";
}

```

0.17 Grouping Increases

```

#include <bits/stdc++.h>
using namespace std;

// Function to calculate minimal penalty
int minPenalty(vector<int>& a) {
    vector<int> lis;
    for (int x : a) {
        auto it = lower_bound(lis.begin(), lis.end(), x);
        if (it == lis.end()) lis.push_back(x);
        else *it = x;
    }
    int LIS = lis.size();
    return max(0, LIS - 2);
}

```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

```

```

    int t;
    cin >> t;
    while (t--) {
        int n;
        cin >> n;
        vector<int> a(n);
        for (int i = 0; i < n; i++) cin >> a[i];
        cout << minPenalty(a) << "\n";
    }
    return 0;
}

```

0.18 8 Queen Problem

```

#include <bits/stdc++.h>
using namespace std;

const int N = 8;

// Check if placing queen at (row,col) is safe
bool isSafe(const vector<int>& queens, int row, int col) {
    for (int r = 0; r < row; r++) {
        int c = queens[r];
        if (c == col) return false; // same
        column
        if (abs(c - col) == abs(r - row)) return false; // same
        diagonal
    }
    return true;
}

// Las Vegas solver for 8 queens
vector<int> lasVegasQueens() {
    vector<int> queens(N, -1); // queens[row] = column of queen
    int row = 0;
    while (row < N) {
        vector<int> validCols;
        for (int col = 0; col < N; col++) {
            if (isSafe(queens, row, col)) validCols.push_back(
                col);
        }
        if (validCols.empty()) {
            // Fail restart from scratch
            return {};
        }
        // Choose random valid column
        int chosen = validCols[rand() % validCols.size()];
        queens[row] = chosen;
        row++;
    }
    return queens;
}

int main() {
    srand(time(0));

    vector<int> solution;
    while (solution.empty()) {

```

```

        solution = lasVegasQueens(); // keep trying until a
            valid solution
    }

    cout << "One solution:\n";
    for (int r = 0; r < N; r++) {
        for (int c = 0; c < N; c++) {
            if (solution[r] == c) cout << "Q ";
            else cout << ". ";
        }
        cout << "\n";
    }
}

```

0.19 Randomized Quicksort

```

#include <bits/stdc++.h>
using namespace std;

mt19937 rng((uint32_t)chrono::steady_clock::now().
    time_since_epoch().count());

int partition_hoare(vector<int>& a, int l, int r) {
    // pick a random pivot and move it to a[l]
    int pidx = uniform_int_distribution<int>(l, r)(rng);
    swap(a[l], a[pidx]);
    int pivot = a[l];

    int i = l - 1, j = r + 1;
    while (true) {
        do { ++i; } while (a[i] < pivot);
        do { --j; } while (a[j] > pivot);
        if (i >= j) return j;
        swap(a[i], a[j]);
    }
}

void quicksort(vector<int>& a, int l, int r) {
    while (l < r) {
        int m = partition_hoare(a, l, r);
        // recurse on smaller side first to keep stack O(log n)
        if (m - l < r - (m + 1)) {
            quicksort(a, l, m);
            l = m + 1;
        } else {
            quicksort(a, m + 1, r);
            r = m;
        }
    }
}

int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    if (n > 0) quicksort(a, 0, n - 1);
}

```

```

for (int i = 0; i < n; ++i) {
    if (i) cout << ' ';
    cout << a[i];
}
cout << '\n';
return 0;
}

```

0.20 Randomized Pi (Monte Carlo)

```

#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>
#include <random>

using namespace std;

#define INTERVAL 10000

int main()
{
    int interval,i;
    double or_dist, pi;
    int circ_p=0, sq_p=0;

    mt19937 mt(time(nullptr));

    for(int i=0;i<100000000;i++)
    {
        double rand_x = static_cast<double>(mt() % (INTERVAL + 1)) /
            INTERVAL;
        double rand_y = static_cast<double>(mt() % (INTERVAL +
            1)) / INTERVAL;

        or_dist= rand_x*rand_x + rand_y*rand_y;

        if(or_dist<=1)
        {
            circ_p++;
        }
        sq_p++;
        //interval++;

        pi= 4.0 * circ_p/sq_p;

        if(i>1000)
            break;
    }

    //cout<<mt() <<endl;
    cout<<pi<<endl;
}

```

```

return 0;
}

```

0.21 DP: Grid Paths

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    const int MOD = 1'000'000'007;
    int n;
    cin >> n;
    vector<string> g(n);
    for (int i = 0; i < n; ++i) cin >> g[i];

    vector<vector<int>> dp(n, vector<int>(n, 0));

    if (g[0][0] == '.' || g[0][0] == '1') dp[0][0] = 1;

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (g[i][j] == '*') {
                dp[i][j] = 0;
                continue;
            }
            if (i == 0 && j == 0) continue;
            long long up = (i > 0) ? dp[i-1][j] : 0;
            long long left = (j > 0) ? dp[i][j-1] : 0;
            dp[i][j] = (up + left) % MOD;
        }
    }

    cout << dp[n-1][n-1] << "\n";
    return 0;
}

```